

架构起步

前言

React 不像 Spring 那样对项目结构有强约束，若缺少统一约定，很容易写出难以维护的散乱代码。本文档介绍 WisePenView 的**前端架构**，帮助新人快速建立「自底向上」的层次概念，并统一设计标准。更细的规范见 `docs/regulations/` 下各设计文档。

架构总览

- **入口**：App.tsx 挂载 `ServiceProvider`（依赖注入）、`ConfigProvider`（主题/语言）、`RouterProvider`（路由）。
- **数据与能力**：所有后端能力通过 **Service 层** 暴露；Service 在 `ServicesContext` 中统一注册，组件通过 `useXxxService()` 获取，不直接 import 具体实现。
- **展示与路由**：路由命中 **Layout**（如 `SystemLayout` / `AuthLayout`），Layout 渲染 **View**（页面）；View 和 **Component** 负责 UI，复杂/复用逻辑抽成组件或 Hook。

项目结构

代码块

```
1  src/
2  |— App.tsx           # 根组件：ServiceProvider + 路由 + 主题
3  |— main.tsx
4  |— router.tsx       # 路由表
5  |— assets/          # 图片等静态资源
6  |— components/      # 可复用 UI 组件（按业务域分子目录）
7  |— constants/       # 全局常量
8  |— contexts/        # React Context，含 ServicesContext (Service 注入)
9  |— hooks/           # 自定义 Hooks
10 |— layouts/          # 布局 (SystemLayout、AuthLayout)
11 |— mocks/           # Mock 实现，与 services 一一对应，MODE=mock 时注入
12 |— services/        # 服务层：接口 + 实现（按领域分子目录）
13 |— store/           # 持久化/全局状态 (Zustand、IndexedDB 等)
14 |— theme/           # 主题配置
15 |— types/           # 领域模型、通用类型
16 |— utils/           # 工具函数 (Axios、response 校验等)
```

自底向上：各层职责

1. Service 层（接口 + 实现 + 依赖注入）

Service 层封装所有与后端的交互，采用 **接口 + 实现 + 依赖注入**，并支持 **Mock 切换**。

目录与文件（以 Auth 为例）：

代码块

```
1  src/services/Auth/
2  |── index.type.ts      # 接口 IAuthService + *Request 等类型
3  |── AuthServices.impl.ts # 实现：用 Axios 调真实 API
4  |── index.ts           # 仅做类型导出，不导出实现
```

- **接口**（`index.type.ts`）：定义 `IAuthService` 和各类 `*Request`，供 Context 与 Mock 约束。
- **实现**（`*Services.impl.ts`）：实现接口，内部使用 `Axios`、`checkResponse` 等；若前后端字段/语义不一致，在实现内做适配，对上只暴露领域语义。
- **Mock**（`src/mocks/Auth/AuthServices.mock.ts`）：同样实现 `IAuthService`，返回本地假数据或 delay；`pnpm mock`（`MODE === 'mock'`）时由 `ServicesContext` 注入 Mock 而非 Impl。

在组件中的使用方式：

- **禁止** 在组件里 `import { AuthServices } from '@services/Auth'` 或直接 import 实现文件。
- **必须** 通过 `useAuthService()` 获取实例（所有 `useXxxService` 在 `src/contexts/ServicesContext.tsx` 中导出）。

代码块

```
1  // 正确
2  const authService = useAuthService();
3  await authService.login(values);
4
5  // 错误：直接引用实现或旧版 AuthServices
6  // import { AuthServicesImpl } from '@services/Auth/AuthServices.impl';
```

这样做的目的：接口集中、易于替换实现（含 Mock）、便于单测与联调前自测；前后端接口差异在实现层消化，视图层只关心「登录、注册」等业务语义。新增服务时需在 `ServicesContext` 完成 7 步注册。

2. Component 层

当某块 UI 较复杂或需多处复用时，抽成**组件**，放在 `src/components/` 下（按业务域分子目录，如 `Group/`、`Drive/`）。

标准三文件：

- `index.tsx`：主逻辑与 JSX
- `index.type.ts`（或 `index.type.tsx`）：Props、组件用到的类型
- `style.module.less`：样式，**禁止内联样式**

Props 即「渲染该组件需要的数据与回调」，用 TypeScript 明确定义。领域实体类型（如 `Group`、`Folder`）倾向于放在 `src/types/`。

能力注入原则：谁用谁拿。需要调 Service 的组件在内部直接 `useXxxService()`；需要某 Hook 的组件直接 `useXxx()`。不通过 props 把 Service 或 Hook 返回值一层层往下传。

3. View 与 Layout

- **View** (`src/views/`)：与路由一一对应，表示「一个可被导航到的页面」。View 可包含简单逻辑；若逻辑复杂或需复用，抽成 Component 或 Hook。
- **Layout**：提供公共壳子。目前有：
 - **AuthLayout**：登录、注册、找回密码等单页。
 - **SystemLayout**：系统内主界面，左侧导航 + 右侧助手 + 中间内容区；中间内容区由子路由渲染对应 View（如 `/app/drive` → `Drive`，`/app/note` → `NotePage`）。

路由在 `src/router.tsx` 中配置；内部系统页使用 `lazy()` 按路由切 chunk，首屏只加载当前路由对应页面。

数据流小结

1. **能力来源**：Service 由 `ServicesContext` 根据环境（开发/生产 vs `MODE=mock`）注入真实实现或 Mock；组件通过 `useXxxService()` 获取。
2. **请求到展示**：用户操作 → View/Component 调用 `useXxxService()` 得到的方法 → Service 实现发请求（或 Mock 返回）→ 组件处理 loading/错误/跳转并渲染。

建议新人按「Service → Component → View & Layout」自底向上过一遍代码。

